

第 0 章 跑通第一个程序

这一章从零开始。目标很简单，把 Python 装到你的电脑上，然后写出并运行你的第一个程序。装环境是最容易劝退新手的一步，所以这一章讲得细一些，包括装完怎么验证、报错了怎么办。这些都弄好之后，后面几章就可以专心写代码了。

0.1 先让电脑说句话

本节任务：装好 Python，让它输出一句话，再用它当计算器算点东西。

0.1.1 第一步，把 Python 装上

Python 不是电脑自带的，得先装。去官网 python.org 的下载页，它会自动认出你的系统，点那个大按钮下载最新版就行。

Windows 用户装的时候有一处要特别注意。安装界面最下面有一个写着 Add python.exe to PATH 的勾选框，**一定要勾上**再点安装。勾了它，你才能在任意位置直接敲 python 这个命令。忘了勾也不要紧，重新运行一遍安装程序，选修改，把它补上即可。

macOS 用户直接下载安装包一路点下去就行。Linux 多数发行版自带 Python，可以先跳到下一步验证一下。

0.1.2 第二步，验证装好了没有

打开命令行。Windows 上按下 Win 加 R，输入 cmd 回车；macOS 上打开终端这个程序。然后输入下面这行并回车：

```
python --version
```

如果看到类似下面的输出，说明装好了，数字是几都没关系，只要是 3 开头：

```
Python 3.13.5
```

0.1.3 第三步，说第一句话

在命令行里直接输入 python 并回车，你会看到几行版本信息，然后出现三个大于号：

```
>>>
```

这就是 Python 在等你输入。试着敲下面这行再回车：

```
>>> print("你好")
你好
```

电脑把这句话打了出来。再试试让它算点东西，输入算式回车就有结果：

```
>>> 2 + 3
5
>>> 65 * 12 + 8
788
```

到这里，Python 已经能用了。想退出这个状态，输入 `exit()` 回车即可。

下面把这几步背后的事情讲清楚。

0.2 Python 到底是什么

本节任务：搞清你刚装的东西是什么，它是怎么把你写的字变成电脑动作的。

你写的 `print("你好")` 是给人看的文字，电脑并不直接懂。中间需要一个翻译，一句一句地读你的代码、照着做，这个翻译就是 **解释器**。你刚才装的 Python，主要就是这个解释器程序。

所以写 Python 程序的过程是这样的，你把想让电脑做的事按 Python 的语法写下来，解释器读一句执行一句。这也是它叫 **解释型语言** 的原因，不需要先整体翻译成另一种文件再运行，写完直接就能跑，改一行也能立刻看到效果。这个特点让它特别适合边学边试。★核心

0.3 两种用法，交互和脚本

本节任务：知道什么时候在命令行里直接敲，什么时候该写成文件。

上一节那种敲一行、立刻出结果的方式，叫 **交互模式**。它的提示符是三个大于号，敲什么立刻算什么，特别适合试探一个想法、验证一句话对不对。缺点是关掉窗口，写的东西就都没了。

真正要写一个完整程序，就得把代码存进文件，这叫 **脚本**。新建一个文本文件，名字随便取，但扩展名要是 `py`，比如 `hello.py`，把代码写进去：★核心

```
1 print("你好，Python")
2 print(2 + 3)
```

存好后，在命令行里先切换到这个文件所在的目录，再运行它：

```
python hello.py
```

```
你好, Python
```

```
5
```

注意脚本和交互模式有个区别。交互模式里你敲 $2 + 3$ 会自动显示 5，但在脚本里，只写一行 $2 + 3$ 屏幕上什么也不会出现，必须用 `print` 明确地要求它输出。这是新手最常见的困惑之一。

两种用法的对照如表 1。往后的练习都是脚本形式，因为要写的代码不止一行。

表 1: 交互模式和脚本的分工。

	交互模式	脚本
怎么进	命令行敲 <code>python</code>	写进 <code>py</code> 文件再运行
适合做什么	试一句、验证想法、当计算器	完整程序、要反复运行的东西
结果怎么出来	敲完自动显示	必须用 <code>print</code>
关掉之后	内容不保留	文件一直在

0.4 代码怎么写才不出错

本节任务：认识 Python 写代码的几条基本规矩，尤其是它很特别的缩进。

0.4.1 一行写一句

Python 一般一行写一条语句，写完直接换行，不需要在末尾加分号。

0.4.2 注释

在代码里加说明文字，用井号开头，井号后面那一行剩下的内容 Python 都不看，只给人读：

```
1 # 这一行是注释, Python 会跳过
2 print("你好") # 也可以写在语句后面
```

养成写注释的习惯，过几天回头看自己的代码时会感谢自己。

0.4.3 缩进决定层次

这是 Python 最有个性的地方。很多语言用大括号表示哪些代码属于一块，Python ★核心用**缩进**，也就是行首的空格。同一块里的代码要缩进相同的距离，标准是四个空格。

```
1 if 3 > 2:
```

```
2     print("这一行属于 if") # 缩进四格，是 if 的一部分
3     print("这一行也属于 if")
4 print("这一行不缩进，不管条件成立与否都会执行")
```

缩进不是排版好看，而是有实际含义。同一块里缩进不一致，程序会直接报错。这一点后面讲分支和循环时会反复用到，现在只要记住行首的空格有意义就够了。

0.4.4 大小写要分清

Python 区分大小写，`print` 和 `Print` 是两个不同的东西，后者会报错。

0.5 用什么写代码

本节任务：挑一个顺手的编辑器，别用记事本。

写一两行可以用交互模式，写完整程序需要一个编辑器。推荐 **VS Code**，免费、通用、装个 Python 扩展就能高亮代码、提示错误、一键运行。装好 VS Code 后，在扩展面板里搜 Python，装微软官方那个即可。

如果暂时不想装，Python 自带一个叫 IDLE 的简易编辑器，在开始菜单里能找到，写小程序够用。

不建议用记事本，它不会高亮代码，也容易在保存时把格式弄乱。

0.6 报错了怎么办

本节任务：新手最常撞见的几种报错，学会看懂它们。

写代码出错是常态，会看报错就成功了一半。报错信息可能有好几行，看不懂上面 ★核心的没关系，**最后一行才是关键**，它告诉你错在哪一类。

下面是三种最常撞见的。

第一种，语法写错了：

```
>>> print("你好"
SyntaxError: '(' was never closed
```

括号只开没关。`SyntaxError` 就是句子没写对，多半是括号引号不成对，或者少了冒号。

第二种，用了不存在的名字：

```
>>> print(nihao)
NameError: name 'nihao' is not defined
```

`NameError` 意思是这个名字它不认识。常见原因是拼错了，或者想写一段文字却忘了加引号。

第三种，缩进不对：

```
>>> if 3 > 2:
...     print("对")
...     print("错")
IndentationError: unindent does not match any outer indentation level
```

`IndentationError` 就是行首空格数量不一致。统一用四个空格就能避开大部分这类问题。

报错不可怕，它其实在帮你定位问题。遇到看不懂的报错，把最后一行原样搜索一下，基本都能找到答案。

0.7 小结

这一章把环境准备好了。Python 是一个解释器程序，读一句执行一句。它有两种用法，交互模式适合试，脚本适合写完整程序，脚本里必须用 `print` 才有输出。写代码要注意缩进有实际含义、大小写要分清。报错时先看最后一行。下一章开始正式写代码，从变量和基本类型讲起。

本章知识点

- **解释器** —— 读一句执行一句的翻译程序，你装的 Python 主要就是它
- **PATH** —— 系统找命令的路径清单，Windows 装 Python 时要勾上加入 PATH
- **交互模式** —— 命令行里敲 `python` 进入，提示符是三个大于号，敲一行出一行结果
- **脚本** —— 把代码存进扩展名为 `py` 的文件，用 `python` 文件名运行
- **print** —— 把结果输出到屏幕，脚本里不写它就看不到东西
- **注释** —— 井号开头的说明文字，只给人看，Python 会跳过
- **缩进** —— 行首的空格，决定哪些代码属于同一块，标准是四个空格
- **SyntaxError** —— 句子写错了，多半是括号引号不成对或少了冒号
- **NameError** —— 用了不认识的名字，多半是拼错或忘了加引号
- **IndentationError** —— 缩进不一致，统一四个空格可避开

第 1 章 变量和基本类型

环境装好了，这一章开始真正写代码。我们做一个小程序，输入几门课的分数，算出平均分并给个等级。麻雀虽小，变量、数字、文字、判断输出这些最基本的东西都会用到。

1.1 先算一份成绩单

本节任务：写一个程序，把三门课的分数存起来，算出总分和平均分，再按平均分给一个等级，最后打印成一份好看的成绩单。

新建一个文件叫 score.py，写入下面的代码：

```
1 name = "小明"
2 chinese = 86
3 math = 92
4 english = 78
5
6 total = chinese + math + english
7 average = total / 3
8
9 print("姓名: ", name)
10 print("总分: ", total)
11 print("平均分: ", average)
```

运行 python score.py，输出是：

```
姓名: 小明
总分: 256
平均分: 85.33333333333333
```

能算了，但平均分那一长串小数很难看。而且我们还想按分数给个等级。把程序改成下面这样：

```
1 name = "小明"
2 chinese, math, english = 86, 92, 78 # 一行赋三个值
3
4 total = chinese + math + english
5 average = total / 3
6
7 if average >= 90:
8     level = "优秀"
9 elif average >= 80:
```

```
10     level = "良好"
11 else:
12     level = "还需努力"
13
14 print(f"{name} 的成绩单")
15 print(f"语文 {chinese} 数学 {math} 英语 {english}")
16 print(f"总分 {total}, 平均分 {average:.1f}, 等级 {level}")
```

这回输出干净多了：

```
小明 的成绩单
语文 86 数学 92 英语 78
总分 256, 平均分 85.3, 等级 良好
```

短短十几行里出现了好几样新东西，变量、两种数字、文字、条件判断、还有那个能把数字塞进文字里的写法。下面逐个讲清楚。

1.2 变量就是给数据起个名字

本节任务：搞懂等号在编程里的含义，以及名字该怎么取。

上面的 `name = "小明"` 里，`name` 是**变量**。变量就是给一份数据起个名字，之后用 **★核心** 这个名字就能取到数据。等号在这里不是数学里的相等，而是**赋值**，意思是把右边的东西放进左边这个名字里。所以下面这行在数学上说不通，在编程里却很常见：

```
1 count = 5
2 count = count + 1 # 先算右边 5+1 得 6, 再放回 count
3 print(count) # 输出 6
```

变量可以随时改成别的值，也可以改成别的类型的值。取名字有几条规矩，必须遵守的是只能用字母、数字和下划线，不能以数字开头，也不能用 Python 已经占用的词比如 `if` 和 `for`。

约定俗成但同样重要的是，名字要能说明这是什么。写 `a` 和 `b` 当时省事，过两天就不知道存的是什么了。多个单词之间用下划线连起来，比如 `total_score` 比 `totalscore` 好读。

1.3 数字有两种

本节任务：分清整数和小数，知道除法为什么总得到小数。

Python 里常用的数字有两种。**整数**就是不带小数点的数，比如 86 和 -3。**浮点数**是带小数点的数，比如 85.3 和 3.14，浮点是小数在计算机里的存法。

运算符和数学课上基本一样，加减乘除写作加号、减号、星号、斜杠。有三个要单独说：

```
1 print(7 / 2) # 2 除法，结果总是小数 3.5
2 print(7 // 2) # 2 整除，只要整数部分 3
3 print(7 % 2) # 2 取余，余数 1
4 print(2 ** 10) # 2 乘方 1024
```

注意 **除法结果总是小数**，即使能除尽也一样， $8 / 2$ 得到的是 4.0 而不是 4。想要整数就用两个斜杠的整除。取余在判断奇偶时很好用，一个数对 2 取余是 0 就是偶数。★核心

浮点数还有一个容易惊讶的地方：

```
1 print(0.1 + 0.2) # 输出 0.30000000000000004
```

这不是 Python 的毛病，而是小数在二进制里存不精确，几乎所有语言都这样。日常使用不用管，只要记住不要直接判断两个小数是否相等，需要精确计算金额时另有专门的办法。

1.4 文字叫字符串

本节任务：会写文字、拼接文字，并把数字漂亮地嵌进文字里。

用引号括起来的文字叫**字符串**。单引号双引号都可以，只要成对。

```
1 a = "小明"
2 b = '你好'
3 c = "他说：'走吧'" # 里外用不同的引号就不会打架
```

字符串能用加号拼起来，用星号重复：

```
1 print("你" + "好") # 你好
2 print("-" * 20) # 打印 20 个短横线，用来分隔很方便
```

但拼接有个麻烦，加号两边必须都是字符串，混进数字就会报错。所以更好的办法是**格式化字符串**，也就是开头那个 f 打头的写法。在引号前面加一个 f，然后把变量写进大括号里：★核心

```
1 name, score = "小明", 85.333
2 print(f"{name} 考了 {score} 分") # 小明 考了 85.333 分
3 print(f"平均分 {score:.1f}") # 平均分 85.3
4 print(f"平均分 {score:.2f}") # 平均分 85.33
```

大括号里冒号后面是格式说明，点一 f 表示保留一位小数，点二 f 就是两位。开头

那份成绩单里的 `average:.1f` 就是这么把长长的小数变短的。这个写法后面会一直用。

1.5 真假和判断

本节任务：认识只有两个取值的布尔类型，并用它做条件判断。

比较两个东西会得到**布尔值**，它只有两个取值，`True` 表示真，`False` 表示假，注意首字母大写。

```
1 print(85 >= 90) # False
2 print(85 >= 80) # True
3 print(3 == 3) # True 两个等号才是判断相等
4 print(3 != 4) # True 不等于
```

这里有个新手必踩的坑，**一个等号是赋值，两个等号才是判断相等**。写成一个等号 ★核心轻则结果不对，重则直接报错。

有了布尔值就能做判断。开头程序里的写法是这样的：

```
1 if average >= 90:
2     level = "优秀"
3 elif average >= 80:
4     level = "良好"
5 else:
6     level = "还需努力"
```

`if` 后面跟条件再跟一个冒号，下一行缩进四格写条件成立时做的事。`elif` 是否则如果，可以有很多个，`else` 是以上都不成立时做的事。Python 从上往下检查，哪个条件先成立就执行哪一块，剩下的都跳过。所以 85.3 会在第二个条件停下，得到良好。

分支还有更多用法，下一章会专门展开，这里够用了。

1.6 类型不对就会出错

本节任务：知道怎么查看和转换类型，避开最常见的一类报错。

每个数据都有类型，用 `type` 可以查：

```
1 print(type(86)) # <class 'int'> 整数
2 print(type(85.3)) # <class 'float'> 浮点数
3 print(type("小明")) # <class 'str'> 字符串
4 print(type(True)) # <class 'bool'> 布尔
```

类型不对的操作会报错，比如字符串加数字：

```
>>> "分数" + 86
TypeError: can only concatenate str (not "int") to str
```

TypeError 就是类型不匹配。解决办法是转换，用 int 转整数，float 转小数，str 转字符串：

```
1 print("分数" + str(86)) # 分数86
2 print(int("86") + 1) # 87
3 print(float("3.5") + 1) # 4.5
4 print(int(3.9)) # 3 直接砍掉小数，不是四舍五入
```

转换在读用户输入时特别重要。**input 拿到的永远是字符串**，哪怕用户输的是数字：★核心

```
1 age = input("请输入年龄：") # 用户输入 18
2 print(age + 1) # 报错，字符串不能加数字
3 print(int(age) + 1) # 19 转成整数才能算
```

这是新手最常撞的一堵墙，看到 TypeError 先想想是不是忘了转换。

1.7 小结

这一章写出了第一个有用的程序。变量是给数据起的名字，等号是赋值不是相等。数字分整数和浮点数，单斜杠除法总得小数，双斜杠整除，百分号取余。文字叫字符串，用 f 打头的格式化写法能把变量和格式一起嵌进去。比较得到布尔值，两个等号才是判断相等。类型不对会报 TypeError，input 拿到的永远是字符串，要算数得先转换。下一章开始处理成批的数据。

本章知识点

- **变量**——给数据起的名字，等号是把右边的值放进左边这个名字
- **整数与浮点数**——不带小数点的和带小数点的，浮点是小数在计算机里的存法
- **除法**——单斜杠结果总是小数，双斜杠只取整数部分，百分号取余数
- **字符串**——引号括起来的文字，单双引号都行，只要成对
- **格式化字符串**——引号前加 f，把变量写进大括号，冒号后可指定保留几位小数
- **布尔值**——只有 True 和 False 两个取值，比较运算的结果
- **一个等号与两个等号**——一个是赋值，两个才是判断相等，新手必踩的坑
- **type**——查看一个数据是什么类型
- **类型转换**——int 转整数，float 转小数，str 转字符串
- **TypeError**——类型不匹配，比如字符串直接加数字
- **input**——读用户输入，拿到的永远是字符串，要算数必须先转换

第 2 章 列表和字典

上一章一门课一个变量，三门课还行，三十门课就没法写了。这一章学怎么把一批数据装在一起，做一个能存很多人的通讯录。

2.1 先做一个通讯录

本节任务：把若干人的姓名和电话存起来，能查、能加、能删、能一次列出全部。

```
1 # 一个人用一个字典，多个人装进一个列表
2 contacts = [
3     {"name": "小明", "phone": "13800001111"},
4     {"name": "小红", "phone": "13900002222"},
5 ]
6
7 # 加一个人
8 contacts.append({"name": "小刚", "phone": "13700003333"})
9
10 # 列出全部
11 print(f"共 {len(contacts)} 人")
12 for person in contacts:
13     print(f" {person['name']} {person['phone']}")
14
15 # 按名字查
16 for person in contacts:
17     if person["name"] == "小红":
18         print("查到了: ", person["phone"])
```

运行结果：

```
共 3 人
小明 13800001111
小红 13900002222
小刚 13700003333
查到了:  13900002222
```

这段代码用了两样东西，方括号的**列表**装多个人，花括号的**字典**装一个人的各项信息。它们是 Python 里最常用的两种容器，下面分别讲。

2.2 列表，一串有顺序的数据

本节任务：会建列表、按位置取值、增删改，并知道切片怎么用。

用方括号把若干数据括起来就是列表，里面装什么类型都行，也可以混着装：

```
1 scores = [86, 92, 78]
2 names = ["小明", "小红"]
3 mixed = [1, "两", 3.0, True]
4 empty = []
```

列表是有顺序的，靠**下标**取值，下标**从 0 开始数**。这是新手最容易错的地方，第 ★核心一个元素的下标是 0 不是 1。负数从末尾倒着数，减一就是最后一个：

```
1 scores = [86, 92, 78]
2 print(scores[0]) # 86 第一个
3 print(scores[2]) # 78 第三个
4 print(scores[-1]) # 78 最后一个
5 print(len(scores)) # 3 一共几个
```

下标超出范围会报 `IndexError`，比如上面这个列表访问 `scores[3]` 就会出错，因为最大下标是 2。

增删改都有对应的写法：

```
1 scores = [86, 92, 78]
2 scores[0] = 90 # 改：把第一个换成 90
3 scores.append(65) # 加：在末尾追加
4 scores.insert(1, 100) # 加：插到下标 1 的位置
5 scores.remove(65) # 删：按值删，删掉第一个 65
6 last = scores.pop() # 删：弹出最后一个并返回它
7 print(scores) # [90, 100, 92]
```

取一段连续的元素叫**切片**，写作方括号里两个下标加冒号，规则是**含头不含尾**： ★核心

```
1 nums = [0, 1, 2, 3, 4, 5]
2 print(nums[1:4]) # [1, 2, 3] 从下标 1 到 3，不含 4
3 print(nums[:3]) # [0, 1, 2] 开头省略表示从头开始
4 print(nums[3:]) # [3, 4, 5] 结尾省略表示一直到末尾
5 print(nums[-2:]) # [4, 5] 最后两个
```

含头不含尾看着别扭，但有个好处，`nums[:3]` 和 `nums[3:]` 正好把列表切成不重不漏的两半。

还有一个和列表很像的东西叫**元组**，用圆括号，区别是建好之后不能改。适合存那

些本来就不该变的东西，比如一个坐标：

```
1 point = (3, 4)
2 print(point[0]) # 3
3 # point[0] = 5 # 这一行会报错，元组不能改
```

2.3 字典，按名字取值

本节任务：会用键值对存一个东西的多项信息，并安全地取值。

列表靠位置取值，可要记住第 3 个是电话第 5 个是地址太累了。**字典**换一种方式，给每个值配一个名字，用花括号写，名字和价值之间用冒号：

```
1 person = {"name": "小明", "phone": "13800001111", "age": 18}
2 print(person["name"]) # 小明
3 print(person["age"]) # 18
```

冒号左边叫**键**，右边叫**值**。键通常用字符串，而且不能重复。

增删改查：

```
1 person = {"name": "小明"}
2 person["phone"] = "138" # 键不存在就是新增
3 person["name"] = "小明明" # 键已存在就是修改
4 del person["phone"] # 删除
5 print("name" in person) # True 判断键在不在
6 print(len(person)) # 1 有几个键值对
```

取一个不存在的键会报 `KeyError`，这是字典最常见的错误。稳妥的做法是用 `get`，★核心键不存在时返回 `None` 而不是报错，还可以指定一个默认值：

```
1 person = {"name": "小明"}
2 # print(person["phone"]) # KeyError
3 print(person.get("phone")) # None
4 print(person.get("phone", "未填")) # 未填
```

遍历字典有三种方式，看你想要什么：

```
1 person = {"name": "小明", "age": 18}
2 for k in person: # 只要键
3     print(k)
4 for v in person.values(): # 只要值
5     print(v)
```

```
6 for k, v in person.items(): # 键和值一起要
7     print(k, "是", v)
```

最后这种 items 的写法最常用，一次拿到键和值。

2.4 两者怎么配合

本节任务：理解开头通讯录为什么是列表套字典这种结构。

回头看开头那个通讯录。一个人有姓名和电话两项信息，各有各的名字，适合用字典。而人有很多个、有先后顺序、要能追加，适合用列表。所以是**列表里装字典**：

```
1 contacts = [
2     {"name": "小明", "phone": "138"},
3     {"name": "小红", "phone": "139"},
4 ]
5 print(contacts[0]) # {'name': '小明', 'phone': '138'}
6 print(contacts[0]["name"]) # 小明 先按位置取人，再按键取信息
```

这种嵌套非常常见，实际项目里从网上取回的数据大多长这样。取值时从外往里一层层来，contacts 取到列表，加下标取到某个人的字典，再加键取到具体信息。

选哪个容器，问自己一个问题就够了。数据是靠位置区分的还是靠名字区分的。靠位置用列表，靠名字用字典，如表 2.1。

表 2.1: 几种容器怎么选。

容器	写法	什么时候用
列表	方括号	一批同类东西，有顺序，要增删
元组	圆括号	一组不该变的值，比如坐标
字典	花括号加冒号	一个东西的多项信息，按名字取

2.5 一个容易踩的坑

本节任务：知道把列表赋给另一个变量时会发生什么。

看这段代码，它的结果多数人第一次都会猜错：

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a) # [1, 2, 3, 4] 动的是 b, a 也变了
```

原因是 `b = a` 并没有复制一份数据，只是给同一个列表起了第二个名字。两个名字 ★核心指向同一份数据，从哪个名字改都是改的它。想真正复制一份，用切片或者 `copy`：

```
1 a = [1, 2, 3]
2 b = a[:] # 或者 b = a.copy()
3 b.append(4)
4 print(a) # [1, 2, 3] 这回 a 没变
5 print(b) # [1, 2, 3, 4]
```

数字和字符串没有这个问题，因为它们不能被改。这个坑在后面处理数据时还会遇到，先有个印象。

2.6 小结

这一章学会了成批地存数据。列表用方括号，靠下标取值，下标从 0 开始，切片含头不含尾，能增删改。元组用圆括号，建好不能改。字典用花括号，靠键取值，取不存在的键会报 `KeyError`，用 `get` 更稳妥，遍历常用 `items`。列表套字典是最常见的组合。要注意把列表赋给另一个变量只是起了别名，改一个另一个也会变。下一章用循环和分支把这些数据真正处理起来。

本章知识点

- 列表——方括号括起来的一串数据，有顺序，可以增删改
- 下标——按位置取值，从 0 开始数，负数从末尾倒数
- `IndexError`——下标超出范围，最大下标是长度减一
- 切片——取一段连续元素，含头不含尾
- 元组——圆括号括起来，建好之后不能改
- 字典——花括号加冒号，用键取值，键不能重复
- `KeyError`——取了不存在的键，用 `get` 可以避免
- `get`——取值时键不存在返回 `None` 或指定的默认值，比方括号稳妥
- `items`——同时遍历键和值，字典最常用的遍历方式
- 列表套字典——一批东西每个都有多项信息时的常用结构
- 别名——把列表赋给另一个变量只是起第二个名字，改一个另一个也变

第 3 章 分支和循环

前面写的程序都是从上到下一遍走完。真正有用的程序需要会拐弯和重复，条件不同走不同的路，同样的事重复很多遍。这一章做一个猜数字游戏，把这两件事讲透。

3.1 先做一个猜数字游戏

本节任务：电脑随机想一个 1 到 100 的数，你来猜，它提示大了还是小了，猜中就结束并报出用了几次。

```
1 import random
2
3 answer = random.randint(1, 100) # 随机想一个数
4 count = 0
5
6 while True: # 一直循环，直到猜中
7     guess = int(input("猜一个 1 到 100 的数: "))
8     count = count + 1
9
10    if guess == answer:
11        print(f"猜中了，你用了 {count} 次")
12        break # 跳出循环，游戏结束
13    elif guess < answer:
14        print("小了")
15    else:
16        print("大了")
```

运行起来是这样，冒号后面的数字是你敲进去的，这一局的答案是 68：

```
猜一个 1 到 100 的数: 50
小了
猜一个 1 到 100 的数: 75
大了
猜一个 1 到 100 的数: 62
小了
猜一个 1 到 100 的数: 68
猜中了，你用了 4 次
```

不到二十行就是一个能玩的游戏。它用到了循环让游戏反复进行，用分支判断大小，还用 break 在猜中时结束。下面逐个拆开。

3.2 分支，让程序会拐弯

本节任务：把条件判断的几种写法弄清楚。

上一章已经见过 if，这里补全。基本形式是 if 跟条件跟冒号，下面缩进四格：

```
1 score = 85
2 if score >= 60:
3     print("及格")
4 else:
5     print("不及格")
```

条件多于两种时用 **elif**，可以写很多个。Python 从上往下检查，谁先成立就执行谁，★核心剩下的一概跳过：

```
1 score = 85
2 if score >= 90:
3     print("优秀")
4 elif score >= 80:
5     print("良好") # 85 在这里停下
6 elif score >= 60:
7     print("及格")
8 else:
9     print("不及格")
```

因为是从上往下且只走一条，条件的顺序很重要。要是把 `score >= 60` 写在最前面，85 分就会被判成及格，后面的分支永远轮不到。

多个条件可以用 **and** 和 **or** 连起来，**not** 表示取反：

```
1 age = 20
2 if age >= 18 and age < 60:
3     print("成年人")
4 if age < 12 or age > 65:
5     print("半价")
6 if not (age >= 18):
7     print("未成年")
```

and 要求两边都成立，**or** 只要有一边成立。上面第一个判断还有更简洁的写法，Python 支持连着写：

```
1 if 18 <= age < 60: # 和 age >= 18 and age < 60 一个意思
2     print("成年人")
```

3.3 for 循环，把一批数据走一遍

本节任务：会用 for 遍历列表和数字区间。

要对一批数据每个都做点事，用 **for**。它的意思是把后面这一批东西一个一个取出来，每取一个执行一遍循环体：

```
1 scores = [86, 92, 78]
2 for s in scores:
3     print(s)
```

s 是个临时名字，每轮循环会自动换成下一个元素。名字可以随便取，取得有意义即可。

想循环固定的次数，用 **range**。range(5) 产生 0 到 4 这五个数，注意从 0 开始且不含终点。

```
1 for i in range(5):
2     print(i) # 依次输出 0 1 2 3 4
3
4 for i in range(1, 6):
5     print(i) # 依次输出 1 2 3 4 5
6
7 for i in range(0, 10, 2):
8     print(i) # 0 2 4 6 8 第三个参数是步长
```

不含终点这一点和切片一致。想数 1 到 5 就写 range(1, 6)。

遍历时如果既要元素又要序号，用 enumerate：

```
1 names = ["小明", "小红"]
2 for i, name in enumerate(names):
3     print(f"第 {i+1} 个是 {name}")
```

配合上一章的字典，遍历通讯录就是这样：

```
1 contacts = [{"name": "小明", "phone": "138"}, {"name": "小红", "phone": "139"}]
2 for person in contacts:
3     print(person["name"], person["phone"])
```

3.4 while 循环，不知道要转多少圈时用

本节任务：分清什么时候用 for 什么时候用 while。

for 适合已知要循环多少次或有一批数据。但猜数字游戏事先不知道要猜几次，这时用 **while**，它的意思是只要条件还成立就一直转：

```
1 count = 0
2 while count < 3:
3     print(count)
4     count = count + 1 # 别忘了改变条件，否则永远转不完
```

用 while 一定要注意，**循环体里必须有能让条件变假的操作**，否则就成了死循环，★核心程序卡住不动。上面那行 count 加一就是干这个的，去掉它程序就会一直打印 0。真遇到死循环，按 Ctrl 加 C 强行终止。

游戏里的写法是 while True，条件永远成立，也就是无限循环，靠里面的 break 跳出。这在不知道循环多少次、只知道满足某条件就结束时很常用。

break 是立刻跳出整个循环，**continue** 是跳过本轮剩下的部分直接进入下一轮：

```
1 for i in range(10):
2     if i == 5:
3         break # 到 5 就整个不循环了，输出 0 1 2 3 4
4     print(i)
5
6 for i in range(5):
7     if i % 2 == 0:
8         continue # 偶数跳过，只输出 1 3
9     print(i)
```

两种循环的分工如表 3.1。

表 3.1: for 和 while 怎么选。

什么时候用	
for	有一批数据要挨个处理，或者要循环固定次数
while	事先不知道多少次，只知道满足某个条件才停

3.5 把循环写短：推导式

本节任务：认识一种从旧列表造新列表的紧凑写法。

从一个列表加工出另一个列表很常见。老实写法是这样：

```
1 scores = [86, 92, 78]
2 plus = []
```

```
3 for s in scores:
4     plus.append(s + 5)
5 print(plus) # [91, 97, 83]
```

Python 提供了一个更短的写法叫**列表推导式**，一行做完同样的事：

```
1 plus = [s + 5 for s in scores]
2 print(plus) # [91, 97, 83]
```

读法是从中间的 for 开始，对 scores 里每个 s，把 s+5 收集起来。还可以在后面加条件，只收集满足条件的：

```
1 scores = [86, 92, 78, 55]
2 passed = [s for s in scores if s >= 60]
3 print(passed) # [86, 92, 78]
```

推导式在数据处理里用得非常多，写熟了比循环清爽。但也别嵌太多层，一行塞不下就老老实实写循环，可读性比短更重要。

3.6 回头看那个游戏

本节任务：把前面的知识对回到开头的代码上。

现在再看开头的游戏，每一行都能讲清楚了：

```
1 import random # 借用标准库来产生随机数
2
3 answer = random.randint(1, 100) # 1 到 100 之间随机取一个，两端都含
4 count = 0 # 记录猜了几次
5
6 while True: # 无限循环，靠 break 结束
7     guess = int(input("猜一个数: ")) # input 拿到字符串，int 转成整数
8     count = count + 1
9
10    if guess == answer: # 从上往下判断，猜中就结束
11        print(f"猜中了，你用了 {count} 次")
12        break
13    elif guess < answer:
14        print("小了")
15    else:
16        print("大了")
```

有两处细节值得留意。一是 `int(input(...))`，上一章说过 `input` 拿到的是字符串，不转成整数就没法和 `answer` 比大小。二是 `random.randint(1, 100)` 两端都包含，和 `range` 不含终点的规矩不一样，用的时候注意区分。

3.7 小结

这一章让程序会拐弯也会重复。分支用 `if elif else`，从上往下检查，谁先成立走谁，所以顺序有讲究。多条件用 `and or not` 连接。`for` 适合遍历一批数据或用 `range` 循环固定次数，`range` 从 0 开始不含终点。`while` 适合事先不知道多少次的情况，一定要保证条件能变假，否则死循环。`break` 跳出整个循环，`continue` 跳过本轮。列表推导式能把简单的加工循环压成一行。下一章把这个游戏拆成几个函数。

本章知识点

- **if elif else** —— 条件分支，从上往下检查，谁先成立执行谁，顺序有讲究
- **and or not** —— 连接多个条件，`and` 要求都成立，`or` 有一个成立即可
- **for** —— 遍历一批数据，每轮自动取出下一个元素
- **range** —— 产生一串数字用于循环，从 0 开始且不含终点
- **enumerate** —— 遍历时同时拿到序号和元素
- **while** —— 条件成立就一直循环，循环体里必须能让条件变假
- **死循环** —— 条件永远为真导致程序卡住，按 `Ctrl + C` 终止
- **break** —— 立刻跳出整个循环
- **continue** —— 跳过本轮剩下的部分，直接进入下一轮
- **列表推导式** —— 用一行从旧列表造出新列表，可以带筛选条件
- **import** —— 借用现成的库，比如 `random` 用来产生随机数

第4章 函数

上一章的游戏能玩，但所有代码挤在一起。想再玩一局就得把整段复制一遍，想改提示语得在好几处改。这一章把它拆成几个函数，顺便学会怎么把代码组织成能反复使用的零件。

4.1 先把游戏拆开

本节任务：把猜数字游戏拆成几个各管一件事的函数，让它能连玩好几局并统计成绩。

```
1 import random
2
3 def make_answer(low=1, high=100):
4     """随机想一个 low 到 high 之间的数"""
5     return random.randint(low, high)
6
7 def judge(guess, answer):
8     """判断猜得如何，返回提示语和是否猜中"""
9     if guess == answer:
10         return "猜中了", True
11     elif guess < answer:
12         return "小了", False
13     else:
14         return "大了", False
15
16 def play_once(low=1, high=100):
17     """玩一局，返回用了多少次"""
18     answer = make_answer(low, high)
19     count = 0
20     while True:
21         guess = int(input(f"猜一个 {low} 到 {high} 的数: "))
22         count = count + 1
23         msg, ok = judge(guess, answer) # 一次收下两个返回值
24         print(msg)
25         if ok:
26             print(f"用了 {count} 次")
27             return count
28
```

```
29 records = []
30 for round_no in range(1, 4): # 连玩三局
31     print(f"--- 第 {round_no} 局 ---")
32     records.append(play_once())
33
34 print(f"三局分别用了 {records} 次，平均 {sum(records)/len(records):.1f} 次")
```

玩完三局的输出大致是：

三局分别用了 [4, 6, 7] 次，平均 5.7 次

和上一章比，代码行数没少多少，好处却很明显。判断逻辑只写在 `judge` 里一处，改提示语只改那里。想改成猜 1 到 1000，调用时传参数即可，不用翻遍代码。玩三局只要循环调用，不用复制粘贴。下面把函数的各个部分讲清楚。

4.2 定义和调用

本节任务：会自己定义函数并调用它。

定义函数用 `def`，后面跟函数名、圆括号、冒号，下一行缩进写函数体：

★核心

```
1 def say_hello():
2     print("你好")
3
4 say_hello() # 调用，输出 你好
```

定义只是把这段代码起个名字存起来，不会执行。写上函数名加圆括号才是**调用**，这时里面的代码才真正运行。圆括号不能省，只写 `say_hello` 不会执行任何东西。

函数名的规矩和变量名一样，建议用动词开头，让人一看就知道它做什么，比如 `make_answer` 比 `answer` 好。

4.3 参数，把数据传进去

本节任务：搞清位置参数、默认参数和关键字参数三种传法。

圆括号里可以写**参数**，调用时把数据传进去：

```
1 def greet(name):
2     print(f"你好，{name}")
3
4 greet("小明") # 你好，小明
5 greet("小红") # 你好，小红
```

同一个函数配上不同的数据就能干不同的活，这是函数最大的价值。

多个参数按顺序对应，这叫**位置参数**：

```
1 def power(base, exp):
2     return base ** exp
3
4 print(power(2, 10)) # 1024 base 是 2, exp 是 10
5 print(power(10, 2)) # 100 顺序换了，含义就变了
```

参数可以给**默认值**，调用时不传就用默认的：

★核心

```
1 def make_answer(low=1, high=100):
2     return random.randint(low, high)
3
4 make_answer() # 用默认的 1 到 100
5 make_answer(1, 10) # 改成 1 到 10
```

参数多了以后按顺序传容易记混，可以写上名字，这叫**关键字参数**，此时顺序无所谓：

```
1 make_answer(high=10, low=1) # 写了名字，顺序随便
```

有个规矩要记住，带默认值的参数必须放在没默认值的后面。写成 `def f(a=1, b)` 会直接报语法错误。

4.4 返回值，把结果拿出来

本节任务：分清打印和返回，会返回多个值。

函数算完的结果要用 **return** 送出来，调用方才能接住。这是新手容易含糊的地方，★核心**打印和返回完全是两回事**。打印只是显示给人看，返回才是把值交给程序继续用：

```
1 def add_print(a, b):
2     print(a + b) # 只显示，不交出去
3
4 def add_return(a, b):
5     return a + b # 交出去
6
7 x = add_print(1, 2) # 屏幕出现 3，但 x 是 None
8 y = add_return(1, 2) # 屏幕什么也没有，但 y 是 3
9 print(x, y) # None 3
```


没写 `return` 的函数，返回的是 `None`，表示没有值。

执行到 `return` 函数就立刻结束，后面的代码不再运行。所以 `return` 也常用来提前退出：

```
1 def check(score):
2     if score < 0:
3         return "分数不能是负的" # 直接结束，不往下走
4     return "分数正常"
```

一次返回多个值，用逗号隔开就行，接的时候也用逗号：

★核心

```
1 def judge(guess, answer):
2     if guess == answer:
3         return "猜中了", True
4     return "没中", False
5
6 msg, ok = judge(50, 68) # 一次接两个
7 print(msg, ok) # 没中 False
```

开头游戏里的 `msg, ok = judge(...)` 就是这么用的。

4.5 函数里的变量出不来

本节任务：理解作用域，避开一类难查的错。

函数里创建的变量只在函数内部有效，出了函数就不存在，这叫**局部变量**：

★核心

```
1 def f():
2     inside = 10
3     print(inside) # 10 在函数内能用
4
5 f()
6 # print(inside) # 报 NameError, 外面不认识它
```

反过来，函数内部可以读到外面的变量，但直接赋值会创建一个同名的新局部变量，不会影响外面的：

```
1 count = 0
2
3 def add():
4     count = count + 1 # 这里会报错
5
```

```
6 # add() # UnboundLocalError
```

这个报错看着奇怪，原因是函数里一旦对 `count` 赋值，Python 就认定 `count` 是局部变量，而等号右边又要先读它，此时它还没有值。正确做法不是去改外面的变量，而是把值传进去、把结果返回来：

```
1 def add(count):
2     return count + 1
3
4 count = 0
5 count = add(count) # 1
```

这样函数只依赖参数、只通过返回值影响外界，谁调用它都能预期结果。养成这个习惯，程序会好懂也好查错。

4.6 给函数写说明

本节任务：用一句话说明函数做什么。

函数体第一行可以写一个字符串，用来说明这个函数干什么，叫**文档字符串**。开头游戏里每个函数下面那行引号就是它：

```
1 def make_answer(low=1, high=100):
2     """随机想一个 low 到 high 之间的数"""
3     return random.randint(low, high)
4
5 print(make_answer.__doc__) # 随机想一个 low 到 high 之间的数
6 help(make_answer) # 也能看到它
```

写不写不影响运行，但函数一多，说明就是自己和别人的救命稻草。一句话讲清做什么即可。

4.7 什么时候该写成函数

本节任务：知道拆函数的时机和标准。

有两个明显信号。一是同样的代码复制了第二遍，二是一段代码长到得往上翻才能看懂。这两种情况都该拆。

拆的标准是**一个函数只做一件事**，名字能概括这件事。判断大小就只判断，不要顺带打印和计数。这样每个函数都短、好测、好改。开头的游戏就是按这个拆的，`make_answer` 只负责出题，`judge` 只负责判断，`play_once` 负责把一局串起来。

拆好之后还有个好处，想改玩法很容易。比如想让电脑来猜你心里的数，只要重写 `play_once` 里问答那部分，`judge` 完全不用动。

4.8 小结

这一章把散落的代码组织成了零件。用 `def` 定义函数，写上名字加圆括号才是调用。参数让同一个函数处理不同数据，可以按位置传、给默认值、也可以写名字传。`return` 把结果交给调用方，和打印是两回事，没写 `return` 就返回 `None`，用逗号可以一次返回多个值。函数里的变量出了函数就不存在，要影响外界应该靠参数和返回值而不是去改外部变量。函数体第一行的字符串是文档说明。代码重复了或者太长了，就该拆成函数，一个函数只做一件事。

到这里，变量、容器、分支循环、函数这几样最基本的东西都齐了，已经足够写出像样的小程序。

本章知识点

- **def** —— 定义函数的关键字，定义本身不执行，调用时才运行
- **调用** —— 函数名加圆括号，圆括号不能省
- **位置参数** —— 按顺序对应的参数，顺序错了含义就变
- **默认参数** —— 给参数预设值，调用时不传就用它，必须放在无默认值参数的后面
- **关键字参数** —— 调用时写上参数名，此时顺序无所谓
- **return** —— 把结果交给调用方，执行到它函数立刻结束
- **打印与返回** —— 打印只给人看，返回才把值交给程序继续用
- **None** —— 没有值，函数没写 `return` 时返回的就是它
- **多返回值** —— 用逗号隔开一次返回多个，接收时也用逗号
- **局部变量** —— 函数里创建的变量，出了函数就不存在
- **文档字符串** —— 函数体第一行的字符串，说明这个函数做什么
- **一个函数只做一件事** —— 拆分函数的标准，名字要能概括这件事

第 5 章 模块和第三方库

前面写的所有代码都是自己从头敲的。实际做事不必这样，很多常见需求别人早就写好了，拿来用就行。这一章统计一段文字里的高频词，全程借用现成的工具，顺便搞清楚这些工具从哪来、怎么装。

5.1 先统计一段文字的高频词

本节任务：给一段英文，找出出现次数最多的几个词，并统计一共有多少个不重复的词。

```
1 import re
2 from collections import Counter
3
4 text = """Python is easy to read and easy to write.
5 Python is widely used in data analysis and machine learning.
6 Read the code, write the code, and read it again."""
7
8 words = re.findall(r"[a-z]+", text.lower()) # 全转小写，只取字母组成的词
9 counter = Counter(words) # 一行完成计数
10
11 print("总词数: ", len(words))
12 print("不重复的词: ", len(counter))
13 print("出现最多的五个: ")
14 for word, n in counter.most_common(5):
15     print(f" {word} {n} 次")
```

运行结果：

```
总词数:  29
不重复的词:  18
出现最多的五个:
  read 3 次
  and 3 次
  python 2 次
  is 2 次
  easy 2 次
```

注意这段代码里真正属于我们的逻辑只有两三行。切词交给了 `re`，计数交给了 ★核心 `Counter`，找前五名用的是 `Counter` 自带的 `most_common`。如果全都自己写，光计数就

得建字典、循环、判断键在不在，排序还得再写一段。**善用现成的工具**是写代码效率的分水岭。

下面讲清楚这些工具是怎么拿到的。

5.2 import 的几种写法

本节任务：会把别人写好的代码引进自己的程序。

一个 py 文件就是一个**模块**。用 **import** 可以把别的模块引进来用，有三种常见写法：

```
1 import math # 引入整个模块，用的时候带模块名
2 print(math.sqrt(16)) # 4.0
3
4 from math import sqrt # 只引入其中一个函数，用的时候不带模块名
5 print(sqrt(16)) # 4.0
6
7 import numpy as np # 引入并起个短名字，长模块名常这么写
```

第一种最稳妥，看到 `math.sqrt` 就知道这个函数是从哪来的。第二种写起来短，但引多了容易和自己的函数重名。第三种给长名字起简称，社区有约定俗成的简称，比如 `numpy` 简称 `np`，见到就照着写，别人一看就懂。

还有一种写法是 `from math import *` 星号，把模块里所有东西都引进来。不建议用，因为不知道引进了什么，很容易把自己的变量覆盖掉。

自己写的文件也能这样引。同一个目录下有个 `tools.py`，里面定义了 `add` 函数，那么在另一个文件里写 `import tools` 再用 `tools.add` 就能调用。这就是把代码分成多个文件管理的方式。

5.3 标准库，装完 Python 就有的工具

本节任务：知道有哪些常用的现成模块，需要时想得起来。

Python 自带一大批模块，不用另外安装，统称**标准库**。常用的如表 5.1。

表 5.1: 几个常用的标准库模块。

模块	干什么
math	数学函数，开方、取整、圆周率
random	随机数、随机抽样、打乱顺序
datetime	日期和时间的计算与格式化
collections	好用的容器，比如上面用的 Counter
json	读写 JSON 格式的数据
pathlib	处理文件路径
re	正则表达式，按模式查找和切分文字

举几个立刻能用上的例子：

```
1 import random
2 print(random.choice(["石头", "剪刀", "布"])) # 随机挑一个
3 nums = [1, 2, 3, 4, 5]
4 random.shuffle(nums) # 原地打乱
5 print(random.sample(range(1, 34), 6)) # 不重复地抽六个
6
7 from datetime import date
8 today = date.today()
9 print(today) # 2026-07-21 这样的格式
10 print(today.year, today.month)
11
12 from collections import Counter
13 print(Counter("hello world").most_common(3)) # 数字符出现次数
```

不用背，知道有这些东西存在就行。需要做某件事时先想想标准库里有没有现成的，搜一下往往就有。

5.4 装别人写的库

本节任务：会用 pip 安装第三方库。

标准库之外还有海量的**第三方库**，需要自己装。装库的工具叫 **pip**，跟着 Python 一起装好了。在命令行里执行：

```
pip install requests
```

几秒后就装好了，之后 import requests 就能用。几个常用命令：

```
pip install 包名 装一个包
pip install 包名==1.2.3 装指定版本
pip uninstall 包名 卸载
pip list 看已经装了哪些
```

国内直接装可能很慢甚至失败，可以换成国内镜像下载：

★核心

```
pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple
```

后面几章要用到的 numpy 和 pandas 就是这样装的。装之前先想清楚是不是真需要，标准库能解决的事没必要引入新依赖。

5.5 虚拟环境，别让项目互相打架

本节任务：理解为什么要给每个项目单独准备一套环境。

所有库都装在一起会有个麻烦。项目甲要某个库的老版本，项目乙要新版本，装了新的甲就跑不了。**虚拟环境**就是给每个项目单独开一个干净的空间，各装各的，互不影响。

创建和使用只有三步：

```
python -m venv .venv 在当前目录建一个虚拟环境
.venv\Scripts\activate 激活，Windows 用这行
source .venv/bin/activate 激活，macOS 和 Linux 用这行
```

激活之后命令行提示符前面会多出一个环境名字，这时 pip install 装的东西都只属于这个项目。用 deactivate 可以退出。

刚入门可以先不用，但知道有这回事。等你同时做两三个项目、或者跟着别人的项目跑代码时，它能省掉很多莫名其妙的报错。项目里常见的 requirements.txt 文件就是记录该项目需要哪些库，别人拿到后一行命令就能装齐：

```
pip install -r requirements.txt
```

5.6 小结

这一章学会了站在别人的肩膀上。一个 py 文件就是一个模块，用 import 引进来，三种写法各有适用场合。标准库装完 Python 就有，math random datetime collections json pathlib re 这几个最常用。标准库之外的第三方库用 pip 装，国内建议配镜像源。虚拟环境给每个项目单独一套依赖，避免版本打架。下一章处理文件，把程序算出来的东西存下来。

本章知识点

- **模块** —— 一个 py 文件就是一个模块，可以被别的文件引用
- **import** —— 引入整个模块，用的时候带模块名，最稳妥
- **from import** —— 只引入其中某个函数，写起来短但容易重名
- **as** —— 给模块起个短名字，长模块名常这么写
- **标准库** —— 装完 Python 就有的一批模块，不用另外安装
- **Counter** —— collections 里的计数容器，配 most_common 直接拿高频项
- **第三方库** —— 标准库之外的库，需要自己装
- **pip** —— 装库的工具，install 装、uninstall 卸、list 看已装
- **镜像源** —— 国内下载慢时换个源，加 -i 参数
- **虚拟环境** —— 给每个项目单独一套依赖，避免版本互相打架
- **requirements.txt** —— 记录项目需要哪些库，一行命令装齐

第 6 章 文件和数据格式

到现在为止，程序一关数据就没了。这一章把第二章的通讯录存进文件，下次打开还能读回来，顺便认识两种最常见的数据格式。

6.1 先把通讯录存下来

本节任务：把通讯录写进文件，程序重启后能原样读回来。

```
1 import json
2 from pathlib import Path
3
4 contacts = [
5     {"name": "小明", "phone": "13800001111"},
6     {"name": "小红", "phone": "13900002222"},
7 ]
8
9 path = Path("contacts.json")
10
11 # 存
12 path.write_text(json.dumps(contacts, ensure_ascii=False, indent=2),
13                 encoding="utf-8")
14
15 # 读
16 loaded = json.loads(path.read_text(encoding="utf-8"))
17 print("读回来", len(loaded), "条")
18 for p in loaded:
19     print(" ", p["name"], p["phone"])
```

运行结果：

```
已存到 D:\code\contacts.json
读回来 2 条
  小明 13800001111
  小红 13900002222
```

打开生成的 contacts.json 看一眼，内容是这样的，人也读得懂：

```
[
  {
```

```
"name": "小明",
"phone": "13800001111"
},
...
]
```

短短几行做完了存和读。下面把文件操作和数据格式分开讲。

6.2 读写文件

本节任务：会打开文件读内容、写内容，并知道为什么要指定编码。

最基本的读写用 `open`，配合 `with` 使用：

```
1 # 写，w 表示写入，原有内容会被覆盖
2 with open("note.txt", "w", encoding="utf-8") as f:
3     f.write("第一行\n")
4     f.write("第二行\n")
5
6 # 读，r 表示读取
7 with open("note.txt", "r", encoding="utf-8") as f:
8     content = f.read()
9     print(content)
10
11 # 一行一行地读
12 with open("note.txt", encoding="utf-8") as f:
13     for line in f:
14         print(line.strip()) # strip 去掉行尾的换行符
```

有两处要特别注意。一是 `with`，它保证文件用完自动关闭，哪怕中间出错也会关。不 ★核心用 `with` 就得自己记着调用 `close`，忘了可能导致内容没真正写进去。二是 `encoding` 要写 `utf-8`，尤其在 Windows 上。不写的话系统可能用别的编码，中文就会变成乱码或直接报错。

几种打开方式的区别：

表 6.1: `open` 的几种模式。

模式	含义
r	读，文件必须已存在
w	写，文件不存在就新建，已存在就清空重写
a	追加，在原有内容末尾接着写

w 会清空原文件，这是新手容易误伤数据的地方。想在末尾添加就用 a。

6.3 路径怎么写

本节任务：会拼路径、判断文件在不在，避开反斜杠的坑。

Windows 的路径用反斜杠分隔，但反斜杠在字符串里有特殊含义，直接写容易出问题。用 `pathlib` 就没这个烦恼，它会自动处理不同系统的差异：

```
1 from pathlib import Path
2
3 p = Path("data") / "contacts.json" # 用斜杠拼接，跨系统通用
4 print(p) # data\contacts.json 或 data/contacts.json
5 print(p.name) # contacts.json
6 print(p.suffix) # .json
7 print(p.exists()) # 文件在不在
8 print(p.resolve()) # 完整的绝对路径
9
10 Path("data").mkdir(exist_ok=True) # 建目录，已存在也不报错
```

`pathlib` 还能直接读写，省掉 `open`：

```
1 Path("note.txt").write_text("你好", encoding="utf-8")
2 print(Path("note.txt").read_text(encoding="utf-8"))
```

小文件用这两个方法最省事，开头存通讯录用的就是它们。

6.4 JSON，存有结构的数据

本节任务：把列表字典这类数据原样存进文件再读回来。

文本文件存的是纯文字，可列表和字典是有结构的。直接把字典写进文件，读回来就成了一串字符，还得自己解析。**JSON** 就是为解决问题而生的格式，它和 Python 的列表字典长得几乎一样。

```
1 import json
2
3 data = {"name": "小明", "scores": [86, 92], "passed": True}
4
5 s = json.dumps(data, ensure_ascii=False) # 转成字符串
6 print(s) # {"name": "小明", "scores": [86, 92], "passed": true}
7
```

```
8 back = json.loads(s) # 再转回来
9 print(back["scores"][0]) # 86
```

记住四个名字就够用了，**dumps** 转成字符串，**loads** 从字符串转回来，带 **s** 的这 ★核心一对处理字符串。还有一对不带 **s** 的直接和文件打交道：

```
1 with open("data.json", "w", encoding="utf-8") as f:
2     json.dump(data, f, ensure_ascii=False, indent=2)
3
4 with open("data.json", encoding="utf-8") as f:
5     back = json.load(f)
```

两个常用参数解释一下。`ensure_ascii` 设成 `False`，中文才会原样存，否则会变成一串看不懂的转义码。`indent` 设成 2 会自动换行缩进，文件打开后人也能读得懂，不写就挤成一行。

JSON 只认识几种类型，字符串、数字、布尔、列表、字典和空值。Python 里的元组会被存成列表，日期这类对象存不了，需要先转成字符串。

6.5 CSV，存表格数据

本节任务：读写表格文件，这是数据处理最常见的格式。

一批结构相同的记录，比如很多行成绩，更适合用 **CSV**。它本质是纯文本，每行一条记录，字段之间用逗号隔开，Excel 能直接打开。

```
1 import csv
2
3 rows = [
4     {"name": "小明", "chinese": 86, "math": 92},
5     {"name": "小红", "chinese": 95, "math": 88},
6 ]
7
8 # 写，newline 设成空字符串是固定写法，不然 Windows 上会多出空行
9 with open("scores.csv", "w", encoding="utf-8-sig", newline="") as f:
10     writer = csv.DictWriter(f, fieldnames=["name", "chinese", "math"])
11     writer.writeheader() # 先写表头
12     writer.writerows(rows)
13
14 # 读
15 with open("scores.csv", encoding="utf-8-sig") as f:
16     for row in csv.DictReader(f):
```

```
17 print(row["name"], row["chinese"])
```

有两个坑要提醒。一是 **newline 要设成空字符串**，否则 Windows 上写出来每行之间会多一个空行。二是编码建议用 **utf-8-sig**，这样用 Excel 打开中文不会乱码。★核心

还有一点，CSV 读出来的所有值都是 **字符串**，哪怕原本是数字。要计算得先转换：

```
1 with open("scores.csv", encoding="utf-8-sig") as f:
2     total = sum(int(row["chinese"]) for row in csv.DictReader(f))
3 print("语文总分", total)
```

这和第一章说的 input 拿到字符串是同一类问题。

两种格式怎么选，看数据形状。有嵌套、每条记录字段可能不一样，用 JSON。一张规整的表，每行字段都相同，用 CSV。

6.6 小结

这一章让数据能留下来。用 open 配 with 读写文件，一定要写 encoding 等于 utf-8，w 模式会清空原文件、追加要用 a。路径用 pathlib 拼接最省心，小文件可以直接用它的 write_text 和 read_text。有结构的数据用 JSON，dumps 和 loads 处理字符串，dump 和 load 处理文件，中文要加 ensure_ascii 等于 False。表格数据用 CSV，注意 newline 设空字符串、编码用 utf-8-sig，而且读出来都是字符串。下一章讲程序出错时怎么办。

本章知识点

- **open** —— 打开文件，r 读、w 写会清空、a 追加
- **with** —— 保证文件用完自动关闭，哪怕中途出错
- **encoding** —— 读写中文一定要写 utf-8，否则容易乱码
- **pathlib** —— 用斜杠拼路径，跨系统通用，还能判断存在与建目录
- **JSON** —— 存有结构的数据，和 Python 的列表字典几乎一样
- **dumps 与 loads** —— 在数据和字符串之间来回转
- **dump 与 load** —— 不带 s 的这一对直接和文件打交道
- **ensure_ascii** —— 设成 False 中文才原样存，否则变成转义码
- **CSV** —— 存规整的表格数据，每行一条记录，Excel 能直接打开
- **newline** —— 写 CSV 时设成空字符串，否则 Windows 上会多空行
- **utf-8-sig** —— 写 CSV 用这个编码，Excel 打开中文不乱码

第7章 出错了怎么办

第零章见过几种报错，但那时只是看看。这一章正面处理错误，让程序遇到意外时不至于整个崩掉，也学会怎么把藏起来的问题找出来。

7.1 先做一个不会崩的计算器

本节任务：写一个把字符串转成数字再相除的函数，用户输入乱七八糟的东西它也不崩，而是给一句人话提示。

```
1 def divide(a, b):
2     """把两个字符串转成数字相除，出问题就返回一句说明"""
3     try:
4         x = float(a)
5         y = float(b)
6         return x / y
7     except ValueError:
8         return "输入的不是数字"
9     except ZeroDivisionError:
10        return "除数不能是零"
11
12 print(divide("10", "4")) # 2.5
13 print(divide("十", "4")) # 输入的不是数字
14 print(divide("10", "0")) # 除数不能是零
15 print(divide("10", "2.5")) # 4.0
```

运行结果：

```
2.5
输入的不是数字
除数不能是零
4.0
```

如果不写 try，第二次调用就会让整个程序停下，后面两行根本不会执行。这就 ★核心是处理错误的意义，不是让错误消失，而是让程序在出错时仍能有个交代、继续往下走。

7.2 try 和 except

本节任务：会捕获错误，并且只捕获自己预料到的那种。

基本结构是 try 里面放可能出错的代码，except 里面放出错后的处理：

```
1 try:
2     n = int("abc")
3 except ValueError:
4     print("转换失败")
```

可以写多个 `except` 分别处理不同的错误，像第一节那样。也可以在一个 `except` 里写多种：

```
1 try:
2     ...
3 except (ValueError, TypeError):
4     print("要么类型不对，要么值不对")
```

想知道错误的具体信息，用 `as` 接住它：

```
1 try:
2     int("abc")
3 except ValueError as e:
4     print("出错了：", e) # 出错了： invalid literal for int() with base 10:
                          'abc'
```

有一个写法要避免。不要用光秃秃的 `except` 接住所有错误：

★ 核心

```
1 try:
2     result = compute()
3 except: # 不好，什么都接住
4     print("出错了")
```

这样写会把所有问题都吞掉，包括你自己写错代码导致的拼写错误。结果是程序不崩了，但行为不对，还查不出原因。正确做法是明确写出预料到的错误类型，没预料到的就让它崩出来，崩掉反而能第一时间发现问题。

完整的结构还有两块：

```
1 try:
2     f = open("data.txt", encoding="utf-8")
3 except FileNotFoundError:
4     print("文件不存在")
5 else:
6     print("读到了", len(f.read()), "个字符") # 没出错才执行
7 finally:
8     print("这句无论如何都会执行") # 常用来做清理
```

7.3 常见错误对照

本节任务：认全最常撞见的几种错误，看到名字就知道大概哪里出了问题。

表 7.1：最常见的几种错误。

错误	通常是什么原因
SyntaxError	语法写错了，括号引号不成对或少了冒号
NameError	用了不存在的名字，多半拼错或忘了加引号
TypeError	类型不对，比如字符串直接加数字
ValueError	类型对但值不合适，比如把汉字转成数字
IndexError	列表下标超出范围
KeyError	字典里没有这个键
ZeroDivisionError	除数是零
FileNotFoundError	要打开的文件不存在
AttributeError	对象没有这个属性或方法，常见于拼错方法名

分清 `TypeError` 和 `ValueError` 对新手很有用。类型不对是 `TypeError`，类型对但值不行是 `ValueError`。比如 `int("abc")` 报的是 `ValueError`，因为传的确实是字符串，只是这个字符串没法变成数字。

7.4 主动抛出错误

本节任务：在参数不合理时，主动报错而不是返回一个奇怪的值。

写函数时如果发现输入不合理，与其硬着头皮算下去，不如直接报错。用 `raise`：

```
1 def set_age(age):
2     if age < 0:
3         raise ValueError("年龄不能是负数")
4     return age
5
6 set_age(-5) # ValueError: 年龄不能是负数
```

这样调用方能立刻知道哪里错了，而不是拿着一个错误的结果继续跑。选哪种错误类型有个简单标准，值不合理用 `ValueError`，类型不对用 `TypeError`。

7.5 怎么把问题找出来

本节任务：程序没报错但结果不对时，用几个笨办法定位问题。

比报错更难缠的是不报错但结果不对。有几个朴素但有效的办法。

第一个是**打印中间结果**。怀疑哪一步就在哪一步打印出来看：

```
1 def average(nums):
2     total = sum(nums)
3     print("调试: total =", total, "个数 =", len(nums)) # 查完记得删掉
4     return total / len(nums)
```

这是最土也最常用的办法，别嫌笨。

第二个是**断言**。用 `assert` 写下你认为必然成立的条件，不成立就立刻报错：

```
1 def average(nums):
2     assert len(nums) > 0, "列表不能是空的"
3     return sum(nums) / len(nums)
4
5 average([]) # AssertionError: 列表不能是空的
```

断言的好处是把问题暴露在源头。没有它的话，空列表要到除法那一步才报 `ZeroDivisionError`，离真正的原因已经隔了一层。

第三个是**缩小范围**。程序很长时，把可疑的那几行单独拎出来，用最小的数据跑一遍。多数问题在这一步就现形了。

第四个是仔细读报错的最后几行。它会指出出错的文件和行号，顺着往上还能看到是从哪里调用过来的。这条调用链常常比错误本身更能说明问题。

7.6 小结

这一章让程序面对意外时更从容。`try` 里放可能出错的代码，`except` 处理预料到的错误，用 `as` 能拿到错误详情，`else` 在没出错时执行，`finally` 无论如何都执行。不要用光秃秃的 `except` 接住一切，那会把自己的 `bug` 也吞掉。分清类型不对的 `TypeError` 和值不合适的 `ValueError`。参数不合理时用 `raise` 主动报错。查问题的四个办法是打印中间结果、写断言、缩小范围、读报错的调用链。下一章学怎么用类把数据和操作打包在一起。

本章知识点

- **try except** —— `try` 里放可能出错的代码，`except` 处理预料到的错误
- **as e** —— 接住错误对象，可以打印出具体原因
- 光秃秃的 **except** —— 接住所有错误的写法，会把自己的 `bug` 也吞掉，不要用
- **else** 与 **finally** —— `else` 在没出错时执行，`finally` 无论如何都执行，常用来清理
- **TypeError** 与 **ValueError** —— 类型不对是前者，类型对但值不合适是后者

- **raise** —— 主动抛出错误，参数不合理时比返回怪值更好
- **打印中间结果** —— 最土也最常用的排查办法
- **assert** —— 断言必然成立的条件，把问题暴露在源头
- **缩小范围** —— 把可疑的几行单独拎出来用最小数据跑
- **调用链** —— 报错最后几行指出文件行号和调用来源，常比错误本身更有用

第 8 章 类和对象

第二章的通讯录是一个列表套字典，加人删人都得写一堆代码，而且谁都能随手改坏里面的数据。这一章用类把数据和操作打包在一起，重写这个通讯录。

8.1 先用类重写通讯录

本节任务：把通讯录做成一个类，数据和增删改查都装在里面，外面用起来只有几行。

```
1 class Contact:
2     """一个联系人"""
3     def __init__(self, name, phone):
4         self.name = name
5         self.phone = phone
6
7     def __repr__(self):
8         return f"{self.name} {self.phone}"
9
10
11 class AddressBook:
12     """一本通讯录"""
13     def __init__(self):
14         self.people = []
15
16     def add(self, name, phone):
17         self.people.append(Contact(name, phone))
18
19     def find(self, name):
20         for p in self.people:
21             if p.name == name:
22                 return p
23         return None
24
25     def remove(self, name):
26         p = self.find(name)
27         if p:
28             self.people.remove(p)
29         return True
```

```
30         return False
31
32     def __len__(self):
33         return len(self.people)
34
35
36 book = AddressBook()
37 book.add("小明", "13800001111")
38 book.add("小红", "13900002222")
39 book.add("小刚", "13700003333")
40
41 print("共", len(book), "人")
42 print("查小红: ", book.find("小红"))
43 print("删小刚: ", book.remove("小刚"))
44 print("现在: ", book.people)
```

运行结果:

```
共 3 人
查小红:  小红 13900002222
删小刚:  True
现在:  [小明 13800001111, 小红 13900002222]
```

和第二章比, 好处在于用起来变简单了。加人是 `book.add`, 查人是 `book.find`, 连 ★核心数人数都能直接用 `len`。怎么存、怎么找这些细节被藏在类里面, 外面不用关心。下面把类的各个部分讲清楚。

8.2 类和实例

本节任务: 分清类和实例, 会定义最简单的类。

类是一个模板, 描述某一类东西长什么样、能做什么。按模板造出来的具体东西叫**实例**。上面 `Contact` 是类, 小明和小红是两个实例。

定义类用 `class`, 类名习惯用大写字母开头:

```
1 class Dog:
2     def bark(self):
3         print("汪")
4
5 d = Dog() # 造一个实例
6 d.bark() # 汪
```

类里定义的函数叫**方法**，第一个参数固定是 **self**，代表调用它的那个实例本身。调 ★核心用时不用传，Python 会自动把实例塞进去。写方法忘了写 **self** 是最常见的错误之一。

8.3 init 和属性

本节任务：让每个实例带上自己的数据。

光有方法还不够，每个实例得有自己的数据。这靠 `__init__` 方法，它在造实例时自动被调用，用来把初始数据装上去：

```
1 class Dog:
2     def __init__(self, name, age):
3         self.name = name # 挂到实例上
4         self.age = age
5
6     def intro(self):
7         print(f"我叫{self.name}, {self.age}岁")
8
9 a = Dog("旺财", 3)
10 b = Dog("小黑", 5)
11 a.intro() # 我叫旺财, 3岁
12 b.intro() # 我叫小黑, 5岁
13 print(a.name) # 旺财 也可以直接取
```

挂在 **self** 上的数据叫**属性**。两个实例各有各的 **name** 和 **age**，互不影响。造实例时写在括号里的参数，就是传给 `__init__` 的。

前后各两条下划线的方法有特殊含义，由 Python 在特定时机自动调用，不用自己写 `__init__` 加括号去调。

8.4 几个有用的特殊方法

本节任务：让自己的类用起来像内置类型一样自然。

除了 `__init__`，还有几个常用的特殊方法。开头的通讯录用了两个。

`__repr__` 决定实例被打印出来的样子。不写的话打印出来是一串没意义的地址：

```
1 class Contact:
2     def __init__(self, name, phone):
3         self.name, self.phone = name, phone
4
```

```
5 c = Contact("小明", "138")
6 print(c) # <__main__.Contact object at 0x000001...> 看不出是谁
```

加上 `__repr__` 就清楚了:

```
1 class Contact:
2     def __init__(self, name, phone):
3         self.name, self.phone = name, phone
4     def __repr__(self):
5         return f"{self.name} {self.phone}"
6
7 print(Contact("小明", "138")) # 小明 138
```

`__len__` 让实例支持 `len`。开头的通讯录写了它，所以能直接 `len(book)`。类似的还有 `__eq__` 决定两个实例怎么算相等，`__getitem__` 让实例支持方括号取值。用到时再查即可。

8.5 继承，站在已有的类上改

本节任务：从已有的类派生出新类，只写不一样的部分。

如果新类和已有的类大部分一样，只有少数不同，用**继承**：

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4     def speak(self):
5         print("...")
6     def intro(self):
7         print(f"我是{self.name}")
8
9 class Dog(Animal): # 括号里写父类
10     def speak(self): # 只改这个方法
11         print("汪")
12
13 class Cat(Animal):
14     def speak(self):
15         print("喵")
16
17 for a in [Dog("旺财"), Cat("咪咪")]:
18     a.intro() # 从 Animal 继承来的
```

```
19 a.speak() # 各自的
```

输出:

```
我是旺财
汪
我是咪咪
喵
```

Dog 和 Cat 没写 `__init__` 和 `intro`, 直接用父类的。speak 各自重写了一份, 这叫**重写**。

如果子类也要有自己的初始化, 记得先调用父类的:

★核心

```
1 class Dog(Animal):
2     def __init__(self, name, breed):
3         super().__init__(name) # 先让父类把 name 装好
4         self.breed = breed
```

`super()` 指代父类。漏掉这一句, 父类里做的初始化就不会执行, `self.name` 也就不存在了。

8.6 什么时候该用类

本节任务: 知道什么情况值得写类, 什么情况没必要。

不是所有代码都该写成类。判断标准是**数据和操作是不是绑在一起的**。

该用类的信号有两个。一是你发现一堆函数总是在操作同一份数据, 比如 `add_contact`、`find_contact`、`remove_contact` 都在摆弄同一个列表, 那这个列表和这些函数就该打包成一个类。二是你需要同时存在好几份互不干扰的同类东西, 比如两本通讯录, 各自有各自的数据。

不该用类的情况也很常见。只是算个数、转换一下格式, 写个函数就够了, 包成类反而绕。第四章说过一个函数只做一件事, 同理, 一个类也应该只描述一样东西。

第二章的做法和这一章的做法都对, 数据少时列表套字典更直接。等到操作变多、需要保证数据不被随手改坏时, 类的好处才显出来。

8.7 小结

这一章把数据和操作打包在了一起。类是模板, 实例是按模板造出来的具体东西。方法的第一个参数固定是 `self`, 代表实例自己。`__init__` 在造实例时自动调用, 用来装上初始数据, 挂在 `self` 上的数据叫属性。`__repr__` 决定打印出来的样子, `__len__`

让实例支持 len。继承能从已有的类派生新类，只写不一样的部分，子类有自己的初始化时要用 super 先调父类。一堆函数总在操作同一份数据，就该考虑写成类。下一章开始接触数据处理，从数组讲起。

本章知识点

- **类**——描述某一类东西长什么样、能做什么的模板，类名首字母大写
- **实例**——按类造出来的具体东西，各有各的数据
- **方法**——类里定义的函数，第一个参数固定是 self
- **self**——代表调用方法的那个实例本身，调用时不用传
- **__init__**——造实例时自动调用，用来装上初始数据
- **属性**——挂在 self 上的数据，每个实例各有一份
- **__repr__**——决定实例打印出来的样子，不写就是一串地址
- **__len__**——让实例支持 len 函数
- **继承**——从已有的类派生新类，只写不一样的部分
- **重写**——子类重新定义父类已有的方法
- **super**——指代父类，子类有自己的初始化时先调它

第9章 数组运算

前面处理一批数字都是用列表加循环。数据一多，循环写起来啰嗦、跑起来也慢。这一章换成 NumPy 的数组，同样的事情能少写很多代码，也快得多。它还是后面所有数据分析和机器学习工具的地基。

装它只要一行：

```
pip install numpy
```

9.1 先算一个班的成绩

本节任务：一个班五名学生三门课的成绩，算出每人总分、每科平均分，并挑出总分最高的人。

```
1 import numpy as np
2
3 names = np.array(["小明", "小红", "小刚", "小美", "小强"])
4 # 每行一个人，每列一门课，依次是语文 数学 英语
5 scores = np.array([[86, 92, 78],
6                     [95, 88, 91],
7                     [55, 61, 48],
8                     [78, 85, 90],
9                     [90, 79, 83]])
10
11 print("形状：", scores.shape)
12 print("每人总分：", scores.sum(axis=1))
13 print("每科平均：", scores.mean(axis=0))
14 print("每人最高分：", scores.max(axis=1))
15
16 best = scores.sum(axis=1).argmax()
17 print("总分最高：", names[best], scores.sum(axis=1)[best])
18 print("及格的人次：", (scores >= 60).sum())
```

运行结果：

```
形状： (5, 3)
每人总分： [256 274 164 253 252]
每科平均： [80.8 81. 78. ]
每人最高分： [92 95 61 90 90]
总分最高： 小红 274
```

及格的人次： 13

注意一个循环都没写。求和、平均、最大值、比较、计数全是一句话。这就是数组的 ★核心价值，把对一批数的操作写成一次运算，而不是一个个遍历。下面把这些用法讲清楚。

9.2 数组和列表有什么不同

本节任务：搞清为什么要用数组而不是列表。

用 `np.array` 可以把列表变成数组：

```
1 import numpy as np
2 a = np.array([1, 2, 3, 4])
3 print(a) # [1 2 3 4]
4 print(type(a)) # <class 'numpy.ndarray'>
```

看着像列表，但有三点关键区别。

第一，数组能整体运算。列表乘 2 是把自己重复一遍，数组乘 2 是每个元素都乘 2：★核心

```
1 print([1, 2, 3] * 2) # [1, 2, 3, 1, 2, 3]
2 print(np.array([1, 2, 3]) * 2) # [2 4 6]
3 print(np.array([1, 2, 3]) + 10) # [11 12 13]
4 print(np.array([1, 2, 3]) ** 2) # [1 4 9]
```

两个等长数组之间也能直接运算，按位置一一对应：

```
1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3 print(a + b) # [11 22 33]
4 print(a * b) # [10 40 90]
```

第二，数组里的元素类型必须一致，全是整数或全是小数。列表可以混装，数组不行。这个限制换来了速度，同类型的数据在内存里排得整整齐齐，运算能一次处理一批。

第三，数组有形状。列表只能是一串，数组可以是二维的表、三维的块。开头那个成绩表就是五行三列的二维数组。

9.3 形状和维度

本节任务：会看形状、改形状，理解按行还是按列计算。

用 `shape` 看形状，`reshape` 改形状：

```
1 a = np.arange(12) # 0 到 11
```

```
2 print(a.shape) # (12,) 一维, 12 个
3 b = a.reshape(3, 4) # 变成 3 行 4 列
4 print(b.shape) # (3, 4)
5 print(b)
```

输出:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

reshape 时元素总数必须对得上, 3 乘 4 等于 12 才行。想让它自己算某一维, 那一维写 -1:

```
1 print(a.reshape(2, -1).shape) # (2, 6) 自动算出 6
```

二维数组求和求平均时, 要指定沿哪个方向算, 这靠 axis 参数。记法很简单, axis=0 是沿着行往下

```
1 b = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(b.sum()) # 21 不指定就是全部加起来
4 print(b.sum(axis=0)) # [5 7 9] 每列的和
5 print(b.sum(axis=1)) # [6 15] 每行的和
```

开头算每人总分用 axis=1, 因为一个人是一行; 算每科平均用 axis=0, 因为一门课是一列。这个参数用错是最常见的问题, 算之前想清楚结果应该有几个数。

9.4 取值和筛选

本节任务: 会从数组里取出想要的部分, 尤其是按条件筛选。

一维取值和列表一样。二维要给两个下标, 先行后列:

```
1 b = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(b[0, 2]) # 3 第 0 行第 2 列
4 print(b[1]) # [4 5 6] 整个第 1 行
5 print(b[:, 0]) # [1 4] 所有行的第 0 列
6 print(b[:, 1:]) # 所有行, 第 1 列往后
```

冒号表示这一维全要, 这个写法在处理表格数据时天天用。

更有用的是按条件筛选。数组和一个数比较, 会逐个元素比较, 得到一个由 True ★核心和 False 组成的数组, 拿它当下标就能选出满足条件的元素:

```
1 a = np.array([86, 92, 55, 78, 90])
2 print(a >= 60) # [ True True False True True]
3 print(a[a >= 60]) # [86 92 78 90] 只留及格的
4 print((a >= 60).sum()) # 4 数一数几个及格
5 print(a[a >= 60].mean())# 86.5 及格者的平均分
```

用 True 和 False 求和能直接数出满足条件的个数，因为 True 当 1 用。开头统计及格人次就是这么算的。

条件可以组合，注意要用 & 和 \，而且每个条件要用括号括起来：

```
1 print(a[(a >= 60) & (a < 90)]) # [86 78] 及格但没到 90 的
```

这里不能用 and 和 or，那两个是给单个真假值用的，数组要用 & 和 竖线。

9.5 广播，形状不同也能算

本节任务：理解形状不一样的数组为什么也能相加。

前面 `np.array([1,2,3]) + 10` 其实就是**广播**。10 是一个数，却和三个元素都相加了，相当于被自动扩展成了 `[10,10,10]`。

二维和一维之间也能广播。比如给每门课加不同的分数：

```
1 scores = np.array([[86, 92, 78],
2                     [95, 88, 91]])
3 bonus = np.array([2, 0, 5]) # 语文加 2，数学不加，英语加 5
4 print(scores + bonus)
```

输出：

```
[[88 92 83]
 [97 88 96]]
```

bonus 只有三个数，却给两行都加上了，它沿着行方向被复制了一遍。广播的规则是从末尾维度开始比对，长度相同或其中一个是 1 就能对上。用不着背规则，实际用的时候形状对不上会报错，报错信息里会写清楚两个形状是什么，照着调整即可。

9.6 常用函数速查

本节任务：认识最常用的一批函数，需要时想得起来。

表 9.1: 常用的数组函数。

写法	干什么
<code>np.zeros(n)</code> 和 <code>np.ones(n)</code>	造一串 0 或 1
<code>np.arange(a, b, step)</code>	像 <code>range</code> 那样造等差数列
<code>np.linspace(a, b, n)</code>	在 <code>a</code> 到 <code>b</code> 之间等分取 <code>n</code> 个数
<code>np.random.rand(n)</code>	<code>n</code> 个 0 到 1 之间的随机数
<code>a.sum()</code> <code>a.mean()</code> <code>a.std()</code>	求和、平均、标准差
<code>a.min()</code> <code>a.max()</code>	最小值、最大值
<code>a.argmax()</code> <code>a.argmin()</code>	最小值、最大值在第几个位置
<code>np.sort(a)</code>	排序
<code>a.T</code>	转置，行列对调

`argmax` 特别有用，它给的是位置而不是值，配合别的数组就能查出对应的信息。开头找总分最高的人就是先 `argmax` 拿到位置，再用这个位置去 `names` 里取名字。

9.7 小结

这一章把成批的数字交给了数组。数组能整体运算，一句话完成过去要循环的事；元素类型必须一致，换来了速度；数组有形状，可以是多维的。`reshape` 改形状，某一维写 -1 让它自己算。求和求平均要指定 `axis`，`axis=0` 得到每列的结果，`axis=1` 得到每行的结果。二维取值先行后列，冒号表示整维全要。用条件筛选能直接选出满足要求的元素，组合条件用 `&` 和竖线并各自加括号。形状不同的数组靠广播也能运算。下一章用表格工具处理带列名的真实数据。

本章知识点

- **数组**——NumPy 的核心结构，能整体运算，元素类型必须一致
- **整体运算**——数组乘 2 是每个元素乘 2，列表乘 2 是重复自己
- **形状**——数组的行列结构，用 `shape` 查看，`reshape` 修改
- **`reshape` 的 -1**——让某一维自动算出来，元素总数必须对得上
- **`axis`**——0 沿行往下走得到每列结果，1 沿列往右走得到每行结果
- **二维取值**——先行后列，冒号表示这一维全要
- **条件筛选**——数组和数比较得到真假数组，拿它当下标选出元素
- **真假求和**——True 当 1 用，求和就是数满足条件的个数
- **组合条件**——用 `&` 和竖线，每个条件要加括号，不能用 `and` 或 `or`
- **广播**——形状不同的数组按规则自动扩展后运算
- **`argmax`**——返回最大值所在的位置而不是值本身

第 10 章 表格和画图

上一章的数组只有数字，列是什么意思得自己记着。真实数据都有列名，还常常缺东少西。这一章用 pandas 处理带列名的表格，再把结果画成图。

装两个库：

```
pip install pandas matplotlib
```

10.1 先分析一份销售数据

本节任务：一份各城市各月的销售记录，算出总额、按城市汇总、找出最好的月份，再把结果画成柱状图。

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "city": ["杭州", "杭州", "北京", "北京", "广州", "广州"],
5     "month": [1, 2, 1, 2, 1, 2],
6     "sales": [120, 150, 200, 180, 90, 130],
7 })
8
9 print(df)
10 print("\n总销售额: ", df["sales"].sum())
11 print("平均每单: ", df["sales"].mean())
12
13 print("\n按城市汇总: ")
14 print(df.groupby("city")["sales"].sum())
15
16 print("\n卖得最好的一条: ")
17 print(df.loc[df["sales"].idxmax()])
```

运行结果：

```
city month sales
0 杭州 1 120
1 杭州 2 150
2 北京 1 200
3 北京 2 180
4 广州 1 90
5 广州 2 130
```

```
总销售额: 870
```

```
平均每单: 145.0
```

```
按城市汇总:
```

```
city
```

```
北京 380
```

```
广州 220
```

```
杭州 270
```

```
Name: sales, dtype: int64
```

```
卖得最好的一条:
```

```
city 北京
```

```
month 1
```

```
sales 200
```

```
Name: 2, dtype: object
```

和上一章比，最大的不同是**每列都有名字**，取数据时写 `df["sales"]` 而不是记住它是 ★核心第几列。分组汇总也只要一行。下面把常用操作过一遍。

10.2 DataFrame，带列名的表

本节任务：会建表、看表、取列取行。

DataFrame 就是一张二维表，每列有名字、有自己的类型。可以从字典建，也可以从文件读：

```
1 import pandas as pd
2
3 df = pd.DataFrame({"name": ["小明", "小红"], "score": [86, 95]})
4
5 df2 = pd.read_csv("scores.csv") # 从上一章存的 CSV 读
6 df.to_csv("out.csv", index=False, encoding="utf-8-sig") # 存回文件
```

读进来先看几眼，这是拿到数据的第一步：

```
1 print(df.head()) # 前五行
2 print(df.shape) # 几行几列
3 print(df.columns) # 有哪些列
4 print(df.dtypes) # 每列是什么类型
5 print(df.describe()) # 数值列的统计摘要，个数 平均 最大最小等
```

```
6 print(df.info()) # 每列有多少非空值，查缺失很有用
```

取列用方括号加列名，取多列传一个列表：

```
1 print(df["score"]) # 一列
2 print(df[["name", "score"]]) # 多列，注意是两层方括号
```

取行有两种写法要分清。loc 按标签取，iloc 按位置取：

★核心

```
1 print(df.loc[0]) # 标签为 0 的那一行
2 print(df.iloc[0]) # 第 0 行
3 print(df.loc[0, "name"]) # 标签 0 行的 name 列
4 print(df.iloc[0:2]) # 前两行
```

刚建的表标签就是 0 1 2，两者看着一样。等筛选过后标签会保留原来的编号，这时两者就不同了。

10.3 筛选和新增列

本节任务：按条件挑出想要的行，并算出新的列。

筛选写法和上一章的数组几乎一样：

```
1 df = pd.DataFrame({"name": ["小明", "小红", "小刚"],
2                      "chinese": [86, 95, 55],
3                      "math": [92, 88, 61]})
4
5 print(df[df["chinese"] >= 60]) # 语文及格的
6 print(df[(df["chinese"] >= 60) & (df["math"] >= 90)]) # 两个条件都满足
```

同样要用 & 和竖线，每个条件加括号。

新增一列直接赋值就行：

```
1 df["total"] = df["chinese"] + df["math"]
2 df["average"] = df[["chinese", "math"]].mean(axis=1)
3 print(df)
```

按某列排序：

```
1 print(df.sort_values("total", ascending=False)) # 总分从高到低
```


10.4 分组汇总

本节任务：按某一列把数据分成几组，each 组各算各的。

这是表格工具最有价值的功能。`groupby` 按某列的取值把行分组，再对每组做统计：

```
1 df = pd.DataFrame({
2     "city": ["杭州", "杭州", "北京", "北京"],
3     "sales": [120, 150, 200, 180],
4 })
5
6 print(df.groupby("city")["sales"].sum()) # 每个城市的总额
7 print(df.groupby("city")["sales"].mean()) # 每个城市的平均
8 print(df.groupby("city").size()) # 每个城市几条记录
9 print(df.groupby("city")["sales"].agg(["sum", "mean", "max"])) # 一次算多个
```

读法是先按 city 分组，再从每组里取 sales 列，再对它求和。理解了这三步，各种变体都能自己写出来。

10.5 处理缺失值

本节任务：真实数据总有空缺，学会查出来并处理掉。

读进来的数据常有空格子，pandas 里显示成 NaN。先查有多少：

```
1 import numpy as np
2 df = pd.DataFrame({"name": ["小明", "小红", "小刚"],
3                     "score": [86, np.nan, 55]})
4
5 print(df.isna().sum()) # 每列有几个缺失
```

处理办法有两类，**丢掉或者填上**：

★ 核心

```
1 print(df.dropna()) # 丢掉有缺失的行
2 print(df.fillna(0)) # 填成 0
3 print(df.fillna(df["score"].mean())) # 填成这一列的平均值
```

选哪种要看情况。数据量大、缺失少，丢掉最省事。缺失多或者每行都珍贵，就得填，填平均值是常见做法。不要不管它，带着 NaN 做计算，结果多半也是 NaN。

10.6 把结果画出来

本节任务：用几行代码把统计结果画成图并存成文件。

pandas 自带画图，底层用的是 matplotlib：

```
1 import matplotlib
2 matplotlib.use("Agg") # 不弹窗，直接存文件
3 import matplotlib.pyplot as plt
4
5 by_city = df.groupby("city")["sales"].sum()
6 by_city.plot(kind="bar") # 柱状图
7 plt.title("Sales by City")
8 plt.tight_layout()
9 plt.savefig("sales.png", dpi=120)
10 print("图已存成 sales.png")
```

kind 换成别的就是别的图，line 折线、barh 横向柱状、hist 直方图、scatter 散点。

有两个实用提醒。第一，在脚本里画图要先设成 Agg 模式，否则某些环境会卡在弹窗上。第二，matplotlib 默认不带中文字体，图里的中文会变成方块，所以上面标题用了英文。要显示中文得指定一个系统里有的中文字体：

```
1 plt.rcParams["font.sans-serif"] = ["SimHei"] # Windows 上的黑体
2 plt.rcParams["axes.unicode_minus"] = False # 负号正常显示
```

10.7 小结

这一章处理了带列名的真实数据。DataFrame 是二维表，每列有名字，read_csv 读、to_csv 存。拿到数据先用 head shape columns dtypes describe info 看几眼。取列用方括号加列名，取行 loc 按标签、iloc 按位置。筛选和数组一样用条件，组合条件要加括号并用 & 和竖线。新增列直接赋值。groupby 分组汇总是最有价值的功能，读法是先分组、再取列、再统计。缺失值先用 isna 查，再决定 dropna 丢掉还是 fillna 填上。画图用 plot 加 kind，脚本里记得设 Agg 模式，中文要另设字体。下一章把这些串成一个完整项目。

本章知识点

- DataFrame —— 带列名的二维表，pandas 的核心结构
- read_csv 与 to_csv —— 从文件读表和把表存成文件
- head 与 info —— 拿到数据先看几行、看每列类型和缺失情况
- describe —— 一次给出数值列的个数、平均、最大最小等摘要
- 取列 —— 方括号加列名，取多列要用两层方括号

- **loc 与 iloc** —— loc 按标签取, iloc 按位置取, 筛选过后两者会不同
- **条件筛选** —— 和数组一样, 组合条件用 & 和竖线并各自加括号
- **新增列** —— 直接给一个新列名赋值即可
- **groupby** —— 按某列分组再对每组统计, 先分组再取列再统计
- **缺失值** —— 显示成 NaN, 用 isna 查, dropna 丢掉或 fillna 填上
- **plot** —— 画图, kind 决定图形种类, 脚本里要先设 Agg 模式
- **中文字体** —— matplotlib 默认不带中文, 要指定字体否则显示成方块

第 11 章 综合项目

前面十一章的东西都是分开学的。这一章把它们串成一个完整的小项目，从造数据到出结论，走一遍真实的分析流程。

11.1 要做的事

本节任务：一家小店有一年的销售流水，我们要回答几个问题：全年卖了多少、哪个品类最赚钱、哪个月最旺、有没有异常的记录。

完整流程分五步，每一步都用到前面某一章的知识：

表 11.1: 项目的五个步骤和对应章节。

步骤	用到什么	来自哪章
准备数据	random 造流水，写成 CSV	第五、六章
读取检查	读 CSV，查缺失和异常	第六、十章
清洗	丢掉或修正坏记录，加派生列	第七、十章
分析	分组汇总，排序	第十章
出结果	打印结论，画图存盘	第十章

11.2 第一步，准备数据

本节任务：造一份带瑕疵的流水数据，存成 CSV。真实数据从来不干净，所以我们故意掺几条问题记录。

```
1 import csv, random
2 from pathlib import Path
3
4 random.seed(42) # 固定种子，每次跑结果一样
5 cats = ["文具", "零食", "饮料"]
6 rows = []
7 for i in range(300):
8     rows.append({
9         "id": i + 1,
10        "month": random.randint(1, 12),
11        "category": random.choice(cats),
12        "amount": random.randint(5, 200),
13    })
```

```
14
15 # 故意掺进三条问题记录
16 rows[10]["amount"] = "" # 缺失
17 rows[50]["amount"] = -30 # 负数
18 rows[80]["category"] = "" # 品类缺失
19
20 with open("sales.csv", "w", encoding="utf-8-sig", newline="") as f:
21     w = csv.DictWriter(f, fieldnames=["id", "month", "category", "amount"])
22     w.writeheader()
23     w.writerows(rows)
24
25 print("已生成", Path("sales.csv").resolve(), "共", len(rows), "条")
```

11.3 第二步，读进来先看看

本节任务：拿到数据不要急着算，先看清它长什么样、有什么毛病。

```
1 import pandas as pd
2
3 df = pd.read_csv("sales.csv", encoding="utf-8-sig")
4 print("形状: ", df.shape)
5 print(df.head(3))
6 print("\n每列缺失数: ")
7 print(df.isna().sum())
8 print("\n金额列的摘要: ")
9 print(df["amount"].describe())
```

摘要里如果最小值是负数，就说明有异常记录。这一步的目的就是把问题找出来，而不是直接进入计算。

11.4 第三步，清洗

本节任务：把坏记录处理掉，再加一列方便后面分析。

```
1 before = len(df)
2
3 df = df.dropna(subset=["amount", "category"]) # 丢掉关键列缺失的行
4 df = df[df["amount"] > 0] # 丢掉金额非正的行
5 df["amount"] = df["amount"].astype(int) # 读进来可能是小数，转成整数
```

```
6
7 def level_of(x):
8     """按金额分档，用第四章的函数写法"""
9     if x >= 150:
10         return "大额"
11     if x >= 50:
12         return "中额"
13     return "小额"
14
15 df["level"] = df["amount"].apply(level_of) # 用函数生成新列
16
17 print(f"清洗前 {before} 条，清洗后 {len(df)} 条，丢掉 {before - len(df)} 条"
18       )
19 print(df.head(3))
```

apply 把一个函数套用到整列的每个值上，这是把自己写的函数用到表格上的常见 ★核心办法，第四章学的函数在这里派上用场。

11.5 第四步，分析

本节任务：回答一开始提出的几个问题。

```
1 print("全年总额：", df["amount"].sum())
2 print("笔均：", round(df["amount"].mean(), 1))
3
4 print("\n各品类总额：")
5 by_cat = df.groupby("category")["amount"].sum().sort_values(ascending=
6     False)
7 print(by_cat)
8
9 print("最赚钱的品类：", by_cat.index[0])
10
11 print("\n各月总额：")
12 by_month = df.groupby("month")["amount"].sum()
13 print("最旺的月份：", by_month.idxmax(), "月，", by_month.max(), "元")
14
15
16 print("\n各档次笔数：")
17 print(df["level"].value_counts())
18
19
20 print("\n品类与档次交叉：")
21 print(pd.crosstab(df["category"], df["level"]))
```

11.6 第五步，出图

本节任务：把两个主要结论画成图存下来。

```
1 import matplotlib
2 matplotlib.use("Agg")
3 import matplotlib.pyplot as plt
4
5 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
6
7 by_cat.plot(kind="bar", ax=axes[0], title="Sales by Category")
8 by_month.plot(kind="line", marker="o", ax=axes[1], title="Sales by Month")
9 axes[1].set_xlabel("month")
10
11 plt.tight_layout()
12 plt.savefig("report.png", dpi=120)
13 print("图已存成 report.png")
```

到这里项目就完成了。从一堆原始记录出发，经过检查、清洗、分析、可视化，得到了能说给别人听的结论。

11.7 这个流程为什么这么排

本节任务：把套路记住，换个数据也能照着做。

五步的顺序不是随便定的。

先看后算这一点最重要。**没检查就直接算，等于在错误的数据上做正确的运算**，结果看着像模像样，实际全错。前面那条负数记录如果不清掉，总额和平均都会被拉低，而你根本不会发现。

清洗要留痕。上面打印了清洗前后的条数，丢掉多少心里有数。如果丢掉了一大半，那多半是清洗规则写得太狠，得回头检查。

分析从粗到细。先看总量，再按维度拆开，最后交叉看。一上来就做复杂的交叉分析，容易在细节里迷路。

结论要能一句话说清。全年多少、哪个品类最好、哪个月最旺，这些是要给人看的。图是帮人快速看懂的工具，不是目的。

换成别的数据，比如学生成绩、网站访问量、天气记录，这套流程照样能用。数据分析的日常工作大部分就是这五步的反复。

11.8 学完这十二章之后

到这里，这份教程就结束了。回头看一下走过的路。

前五章打地基，变量和类型、列表和字典、分支和循环、函数，这些是任何程序的骨架。中间三章让程序能用起来，借用现成的库、把数据存进文件、出错时不崩溃。第八章用类把数据和操作打包。最后三章转向数据，数组、表格、完整项目。

接下来往哪走，看你想做什么。想做数据分析，继续深入 pandas 和统计知识。想做网站后端，去学 Flask 或 FastAPI。想做自动化脚本，标准库里的 os 和 subprocess 值得细看。想往人工智能方向走，这一章的数组和表格正好是那边的入口，深度学习框架里的张量就是数组的延伸。

不管往哪走，方法都一样，找一个具体的小目标，动手做，卡住了就查，做完再回头补原理。这份教程每一章都是这么安排的，往后也这么学就行。

本章知识点

- **五步流程** —— 准备数据、读取检查、清洗、分析、出结果
- **先看后算** —— 没检查就算，等于在错误的数据上做正确的运算
- **清洗留痕** —— 打印清洗前后的条数，丢太多说明规则写狠了
- **apply** —— 把自己写的函数套用到整列的每个值上
- **value_counts** —— 数一列里每个取值出现多少次
- **crosstab** —— 交叉统计两个维度
- **idxmax** —— 找出最大值对应的标签，比如哪个月最旺
- **固定随机种子** —— 让每次运行结果一样，便于核对
- **从粗到细** —— 先看总量，再按维度拆，最后交叉看